

Тема урока: абстрактные классы, модуль abc

1. Абстрактные классы
2. Модуль `abc`
3. Модуль `collections.abc`

Аннотация. Урок посвящен абстрактным классам и модулям `abc` и `collections.abc`.

Абстрактные классы

Рассмотрим иерархию классов:

```
class Animal:
    def sound(self):
        pass

    def move(self):
        pass

class Cat(Animal):
    def sound(self):
        return 'мяу'

    def move(self):
        return ''
```

Базовый класс `Animal` не представляет никакого конкретного животного, а просто задает общий интерфейс, присущий всем животным. Несмотря на это, мы можем создать экземпляр класса `Animal`.

Приведенный ниже код:

```
animal = Animal()

print(animal.sound())
print(animal.move())
```

выполняется без ошибок, хотя и не имеет большого смысла.

Для ситуаций, в которых не имеет смысла создавать экземпляры базового класса, используют **абстрактные классы**.

Абстрактный класс – очень важная концепция объектно-ориентированного программирования. Это хорошая практика принципа "не повторяйся". Абстрактные классы очень удобно использовать для определения общего интерфейса для различных реализаций.

Абстрактный класс имеет некоторые особенности:

- абстрактный класс не содержит всех реализаций методов, необходимых для полной работы это означает, что он содержит один или несколько **абстрактных методов**
- абстрактный метод – это только объявление метода, без его подробной реализации
- абстрактный класс предоставляет интерфейс для наследников, чтобы избежать дублирования кода, при этом нет смысла создавать экземпляр абстрактного класса
- классы наследники должны реализовать **все** абстрактные методы для создания конкретного класса, который соответствует интерфейсу, определенному абстрактным классом

Другими словами, абстрактный класс определяет общий интерфейс для набора подклассов. Он предоставляет общие атрибуты и методы для всех подклассов, чтобы уменьшить дублирование кода. Он также заставляет подклассы реализовывать абстрактные методы, чтобы избежать каких-то несоответствий.

Для создания абстрактных классов используется встроенный модуль `abc`. Абстрактный класс можно определить с помощью наследования от класса `abc.ABC`, а абстрактный метод – с помощью декоратора `@abc.abstractmethod`.



ABC – это аббревиатура от слов Abstract Base Class (Абстрактный Базовый Класс).

В приведенном ниже коде мы объявляем абстрактный класс `Animal`, содержащий два абстрактных метода `move()` и `sound()`:

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def move(self):
        pass

    @abstractmethod
    def sound(self):
        pass
```

Python не разрешает создавать экземпляры абстрактных классов. При попытке сделать это будет возбуждено исключение

Приведенный ниже код:

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def move(self):
        pass

    @abstractmethod
    def sound(self):
        pass

animal = Animal()
```

приводит к возбуждению исключения:

```
TypeError: Can't instantiate abstract class Animal with abstract methods move,
sound
```

Обратите внимание, что класс является абстрактным только в том случае, если он наследуется от класса `abc.ABC` и имеет хотя бы один абстрактный метод. В противном случае класс не будет считаться абстрактным, и его экземпляр можно будет создать.



Наследник абстрактного класса должен переопределить **все** абстрактные методы, иначе экземпляр такого класса будет невозможно создать.

На самом деле абстрактный метод в Python не обязательно должен быть полностью абстрактным, что отличается от некоторых других объектно-ориентированных языков программирования. Можно определить некоторый общий функционал в абстрактном методе и использовать функцию `super()` для вызова его в подклассах.

Приведенный ниже код:

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        print('Не определен')

    @abstractmethod
    def move(self):
        print('Животное движется')

class Cat(Animal):
    def sound(self):
        print('мяу')

    def move(self):
        super().move()
        print('Кот движется')

cat = Cat()

cat.move()
cat.sound()
```

ВЫВОДИТ:

```
Животное движется
Кот движется
мяу
```

Как показано в приведенном выше примере, абстрактный метод `move()` может содержать некоторый функционал и может вызываться подклассом с помощью `super()`. Несмотря на реализацию в базовом классе, метод `move()` является все еще абстрактным методом, и нам необходимо полностью реализовать (переопределить или дополнить) его в подклассах.

Совместно с декоратором `@abstractmethod` можно использовать такие декораторы, как `@property`, `@classmethod` и `@staticmethod`, при этом декоратор `@abstractmethod` следует применять как самый внутренний декоратор.

Определение абстрактного метода класса:

```
from abc import ABC, abstractmethod

class C(ABC):
    @classmethod
    @abstractmethod
    def abstract_class_method(cls, ...):
        ...
```

Определение абстрактного статического метода:

```
from abc import ABC, abstractmethod

class C(ABC):
    @staticmethod
    @abstractmethod
    def abstract_static_method(...):
        ...
```

Определение абстрактного свойства только для чтения:

```
from abc import ABC, abstractmethod

class C(ABC):
    @property
    @abstractmethod
    def abstract_property(self):
        ...
```

Также можно определить абстрактное свойство для чтения и записи, соответствующим образом пометив один или несколько базовых методов как абстрактные:

```
from abc import ABC, abstractmethod

class C(ABC):
    @property
    def x(self):
        ...

    @x.setter
    @abstractmethod
    def x(self, value):
        ...
```

Если только некоторые компоненты являются абстрактными, только эти компоненты необходимо обновить, чтобы создать конкретное свойство в подклассе:

```
class D(C):
    @C.x.setter
    def x(self, value):
        ...
```

Примечания

Примечание 1. Если класс не реализует какой-либо метод, то вместо заглушки `pass` он может возбуждать исключение `NotImplementedError` или содержать строку документации.

```
from abc import ABC, abstractmethod

class C(ABC):
    @abstractmethod
    def method1(self, input):
        '''docstring'''

    @abstractmethod
    def method2(self, output, data):
        raise NotImplementedError
```

Примечание 2. Абстрактные классы могут содержать не только абстрактные методы, но и обычные. Такие методы не обязаны быть переопределены дочерними классами.

Приведенный ниже код:

```
from abc import ABC, abstractmethod

class C(ABC):
    @abstractmethod
    def method1(self):
        pass

    def method2(self):
        return 2
```

